

Advanced Git practical

Nick Brown

12 July 2016

1 Introduction

The purpose of these practicals is to gain more advanced experience using Git. The simple exercises in the Git practical will be sufficient for the vast majority of your needs. The material here is mainly aimed at making your use of Git more productive - especially when multiple people are working with the same repository.

2 Branching and merging

Branching in Git is great - it's efficient, simple and lightweight. Unlike more centralised revision control software such as CVS or Subversion, Git entirely separates the concepts of local and remote branches. This is important, as it allows you to create many local, private, branches (in your local repository) and push to entirely different remote branches. For example, you might be working on a number of features and bug fixes, each of which can be kept entirely separate by creating individual local branches for them. Once complete, you can then push these branches up to a single (often master) remote branch.

First, create a new branch using the `git branch` command, i.e. `git branch newbranch` which will branch from where you currently are. Without any arguments, the command will show us what branch we are on

```
$ git branch  
  
* master  
  
newbranch
```

The asterisk means that we are currently on the `master` branch, to move to our newly created branch use the `git checkout` command

```
$ git checkout newbranch  
  
Switched to branch 'newbranch'
```

Create a new file (I have called mine `test.c`) with some text in it, add it and commit that file - this will be committed to the `newbranch` branch. Go back to the `master` branch via `git checkout master` and you will see that that new file no longer exists. We have two branches, a `master` branch and `newbranch` which contains this additional new file that we created and this pattern can be extended across a number of

branches all of them separate. Now let's imagine that we have finished with `newbranch` and wish to merge its contents back into the `master` branch.

```
$ git checkout master

$ git merge newbranch

Updating 2b84fa8..00e0ac5

Fast-forward
 test.c |    1 +
 1 files changed, 1 insertions(+), 0 deletions(-)
 create mode 100644 test.c
```

Whilst the `merge` command is a very useful one, there is also the `git rebase` command, which take all the changes that were committed on one branch and replay them on another one. In the example above, instead of `git merge newbranch` we could have done `git rebase newbranch`. It is a common workflow to merge (or rebase) local temporary branches back to the `master` branch before pushing to the remote repository. You can delete local branches via `git branch -D newbranch` which is useful to keep your local repository tidy once you have finished with a branch.

3 Stashing

Imagine the scenario - you are currently working on some code, which is far from ready to be checked in, and suddenly a high priority bug report comes in which you have to sort out immediately. We might have numerous uncommitted files that we want to put to one side for the moment, probably switch branches, and then later on when the bug is fixed come back to what we were doing. Stashing allows us to pause what we are currently working on, do something else, and come back to it later.

Picking up from the last example, edit `test.c` to add some extra text to it and verify the files state via `git status` and/or `git diff`. Now issue `git stash` and try `git status` or `git diff` again - the changes have gone - but don't worry they are not lost! To get our modifications back just issue `git stash apply`. The `stash` command actually acts like a stack where you can stash multiple layers of modifications. One can view the stash stack via `git stash list` and un-pause the entry at the top of the stack via `git stash pop` as well as clearing the entire thing using `git stash clear`.

4 Tagging

Tagging is a common way to denote specific versions of your code (for instance a release), or perhaps even exactly one commit for some reason. We do this via the `git tag` command.

```
$ git tag -a release1 -m "First release"
```

We have a number of views that we can take of tags - firstly the `git show` command will provide some general information i.e. `git show release1`. To figure out how far back a tag is in our commit history, then we can use `git describe`. Issue `git describe --tags`, now make some changes and commits to your Git repository and issue that same command again - you will see the output has changed describing the tag id, number of commits that have occurred since the tag was created and abbreviated latest commit id.

5 Going back in time

There are plenty of ways to go back to previous commit versions. You can explicitly reference versions via the hash reported via `git log`.

```
$ git log

commit 4a155e5b3b4548f5f8139b5210b9bb477fa549de
Author: John Doe <Doe.John.10@gmail.com>
Date:   Thu Jul 21 20:51:38 2011 -0500

$ git checkout 4a155e5

Note: checking out '4a155e5'.
```

See that we use the first seven characters of the commit id in the `checkout` command. To go back to where you were, just issue `git checkout master` which will reset to the latest commit.

Some people find interactive rebases easier to handle, for example issue the command `git rebase -i HEAD~4`. Your editor will open and through this you can, in this case, the previous four commits. Interactive rebases allow you to edit, reorder and combine previous commits.

6 Ignoring stuff

You don't always want everything in your Git repository to be under source control. As a general rule; if it can be generated, then leave it out. This would apply to things like binary files, external libraries, auto-generated code and temporary files that editors love to create. Git looks for a `.gitignore` file which allows you to select which files you choose to ignore and it effects all directories in your project; unless they include their own `.gitignore` file too. An example `.gitignore` file is in this section.

```
Can ignore specific files
bin/myexecutable

# Use wildcards as well
*.so
*.exe

# Can also ignore all directories and files in a directory.
tmp/**/*
```

7 Summary

We have looked at some more advanced commands and uses that Git has to offer. If you go on to work in software engineering, and your projects use Git, then the items discussed here are likely to be very useful to you! As per the Git practical, if you can then I would always suggest installing `git gui` which provides you with a visualisation of the repository state and saves you having to remember the exact syntax of commands.

For further information, the Git man pages are very informative, additionally, if you issue the `git help [command]` (where `[command]`) is replaced with whatever command you want more information about, then the specific man page for that command will be displayed.

Another recommended source for further information can be found at <http://git-scm.com/documentation> which recaps in detail the topics discussed in these practicals.